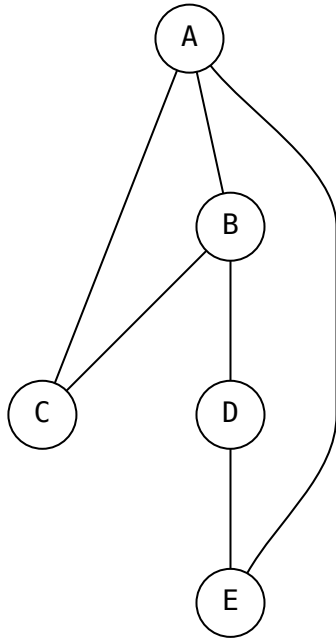


ITC MPSI

Notes de cours sur les graphes

Les *graphes* sont une manière, en informatique, de représenter des données avec des liens.

Exemple 1



I) Présentation générale

I.1) Définitions et vocabulaire

Définition 2 [Graphes]

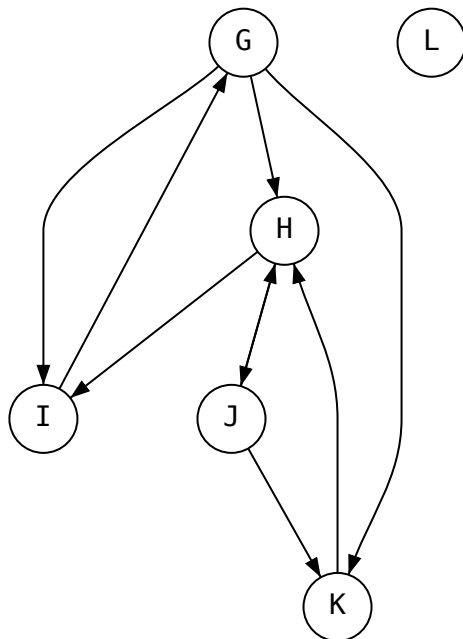
Un *graphe* est un couple $G = (S, A)$, où S est un ensemble fini de *sommets* (ou *noeuds*) et $A \in S \times S$ est l'ensemble des *arrêtes*. Chaque arrête relie deux sommets *distincts*.

On dit que le graphe est *orienté* si les arrêtes ont un sens ("que ce sont des flèches"). On les appelle alors souvent *arcs*.

Définition 3 [Voisin]

Si on a une arrête (s, s') , on dit que s' est un voisin de s .

Exemple 4



Ici, les voisins de H sont J et I.

Définition 5 [Degrés d'un sommet]

Dans un graphe non orienté, le degré d'un sommet s , noté $d(s)$, est le nombre d'arrêtes qui le touchent.

Dans un graphe orienté, on note $d_+(s)$ le degré entrant et $d_-(s)$ le degré sortant, qui sont respectivement le nombre d'arcs qui arrivent et qui partent de s .

Exemple 6

- $d(B) = 3, d(D) = 2$
- $d_+(G) = 1, d_-(G) = 3, d_+(L) = 0$

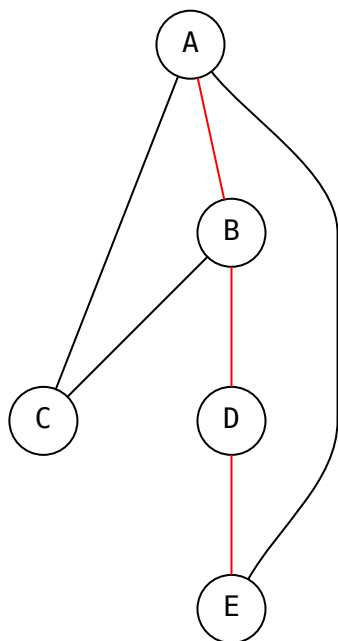
II) Chemins, cycles et connexité

Définition 7 [Chemin]

Un *chemin* dans un graphe est une suite finie d'arrêtes consécutives $(s_1, s_2), (s_2, s_3), \dots, (s_{n-1}, s_n)$.

On parle de chemin *élémentaire* lorsque les sommets $s_1, \dots, s_n$ sont distincts, et de chemin *simple* lorsque les arrêtes sont distinctes (ces mots de vocabulaire ne sont pas au exigibles).

Exemple 8

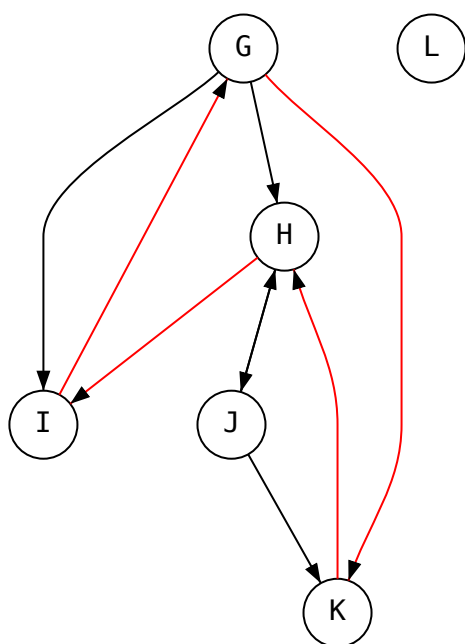


AB, BD, DE est un chemin.

Définition 9 [Cycle]

Un cycle est un chemin simple dont le premier et le dernier sommet sont les mêmes.

Exemple 10



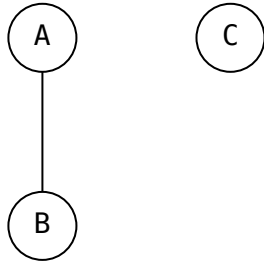
HI, IG, GK, KH est un cycle.

Définition 11 [Connexité]

Un graphe non orienté est *connexe* si, pour tout sommets s_1, s_2 , il existe un chemin de s_1 à s_2 .

Exemple 12

L'exemple 1 est connexe.



Ce graphe n'est pas connexe.

Remarque 13

La notion de connexité est hors-programme pour les graphes orientés. Pour donner une idée, on parlera de graphe *fortement connexe* si il existe un chemin de s_1 à s_2 et de s_2 à s_1 .

III) Représentation informatique

Il y a plusieurs manières de représenter les graphes en Python. On va en voir deux principales.

III.1) Représentation par liste d'adjacence

L'idée ici, est de stocker, pour chaque sommet, l'ensemble de ses voisins. Dans le cas des graphes non-orienté, on stockera en général les connections dans les deux sens.

Par exemple, pour le graphe de l'ex 1, la liste d'adjacence du sommet A est :

```
['B', 'C', 'E']
```

Ensuite, on va regrouper toutes ces listes d'adjacence dans un dictionnaire, pour avoir la représentation suivante :

```
g = {
    'A': ['B', 'C', 'E'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B'],
    'D': ['B', 'E'],
    'E': ['A', 'D']}
}
```

Exercice : faire la même chose avec l'exemple 2.

Remarque 14

Dans le cas où les sommets sont numérotés de 0 à $n-1$, on peut utiliser une liste à la place d'un dictionnaire.

Remarque 15

La représentation par liste d'adjacence est très adaptée lorsqu'on veut souvent accéder aux voisins d'un sommet particulier, par exemple lors d'une exploration du graphe.

III.2) Matrice d'adjacence

Dans le cas de la matrice d'adjacence, on va toujours numéroter les sommets de 0 à $n - 1$.

On va alors se donner une matrice de taille $n \times n$, et à la position (i, j) , on va placer 1 si l'arrête $i \rightarrow j$ si elle existe, et 0 sinon.

Dans le cas de l'exemple 1 :

```
g = [[0, 1, 1, 0, 1],  
      [1, 0, 1, 1, 0],  
      [1, 1, 0, 0, 0],  
      [0, 1, 0, 0, 1],  
      [1, 0, 0, 1, 0]]
```

Remarque 16

La matrice d'adjacence d'un graphe non-orienté est symétrique.

Exercice : faire la même chose avec l'exemple 2.

Remarque 17

La représentation par matrice d'adjacence est adaptée lorsqu'on a souvent besoin de tester si deux sommets sont adjacents.

IV) Pondération

IV.1) Applications

Les graphes se retrouvent un peu partout en informatique :

- Pour tout ce qui est carte. Une carte de transports en commun est un graphe par exemple.
- Internet, à deux niveaux :
 - l'architecture physique d'internet est un grand graphe, avec des serveurs, des routeurs et des terminaux reliés par de la fibre optique, des ondes, etc
 - le web : les pages webs, et les liens entre elles, sont bien représentées par des graphes.

Dans tous ces exemples, des algorithmes sur les graphes (on en étudiera quelques uns) sont utilisés. Par exemple, la recherche d'itinéraire de votre appli de carte préférée, le programme qui décide comment acheminer votre message whatsapp, ou encore les moteurs de recherche (Google).

Dans beaucoup de ces applications, toutes les arrêtes ne sont pas égales. Par exemple, un trajet en train peut prendre deux fois plus longtemps qu'un autre.

IV.2) Graphes pondérés

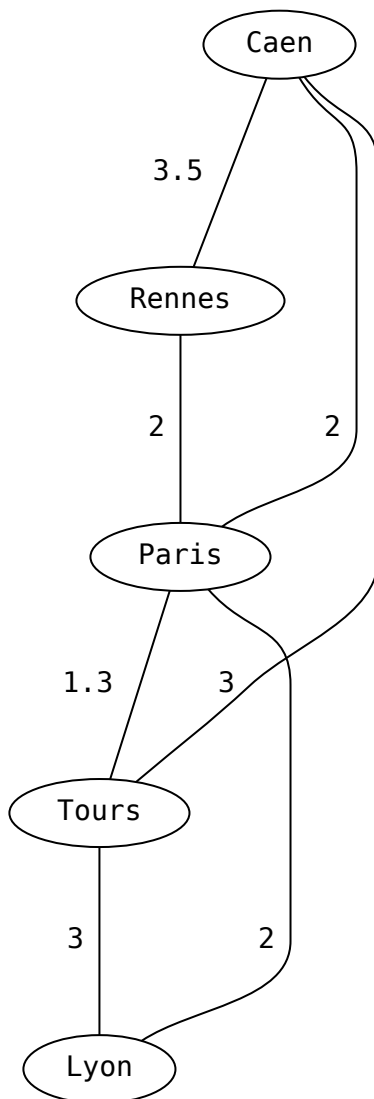
Pour représenter ces différences, on va introduire la *pondération*.

Définition 18 [Graphe pondéré]

Un graphe pondéré est un graphe $G = (S, A)$, orienté ou non, auquel on associe une fonction de pondération $\omega : A \rightarrow E$ qui associe un *poids* (ou étiquette) à chaque arrête.

E est ici un ensemble quelconque, mais en pratique, ce sera souvent $\mathbb{N}$ ou $\mathbb{R}^+$.

Exemple 19



Exemple de quelques villes et le temps de trajet en train entre elles. Quels sont les chemins possibles entre Caen et Lyon ? Quel est leur poids ?

Remarque 20

On peut alors parler du poids d'un chemin, d'un cycle, ...

IV.3) Implémentation des graphes pondérés

IV.3.1) Liste d'adjacence

On peut stocker les poids directement dans la liste d'adjacence, en mettant des couples (voisin, poids) au lieu des simples voisins :

Exemple 21

```
g = {
    'Caen': [('Rennes', 3.5), ('Tours', 3.0), ('Paris', 2.0)],
    'Rennes': [('Caen', 3.5), ('Paris', 2.0)],
    'Paris': [('Caen', 2.0), ('Rennes', 2.0), ('Tours', 1.3), ('Lyon', 2.0)],
    'Tours': [('Caen', 3.0), ('Paris', 1.3), ('Lyon', 3.0)],
    'Lyon': [('Paris', 2.0), ('Tours', 3.0)]
}
```

IV.3.2) Matrice d'adjacence

On peut mettre le poids de l'arrête à la place de "1", et utiliser une valeur qui n'est pas dans les poids à la place de "0" (par exemple $+\infty$, où, si ce n'est pas une option, -1).

Exemple 22

```
g = [[ 0.0, 3.5, 2.0, 3.0, -1.0],
     [ 3.5, 0.0, 2.0, -1.0, -1.0],
     [ 2.0, 2.0, 0.0, 1.3, 2.0],
     [ 3.0, -1.0, 1.3, 0.0, 3.0],
     [-1.0, -1.0, 2.0, 3.0, 0.0]]
```

V) Parenthèse : structures de données, pile, file, file de priorité

V.1) Structure de données abstraite

Une structure de donnée est un objet informatique permettant de stocker une collection d'objets. Vous en connaissez déjà deux : les listes et les dictionnaires.

On distingue la *structure de donnée* de son *implémentation* : lorsque vous utilisez une liste ou un dictionnaire en python, vous utilisez par exemple la méthode `.append(...)`, ou bien l'opérateur `d[obj]`. En revanche, vous ne vous intéressez pas à ce qu'il se passe, concrètement, quand vous les utilisez : ils fonctionnent et c'est ce qui compte.

Dans ce cas, vous utilisez la structure de donnée, sans vous soucier de son implémentation.

Définition 23 [Structure de donnée abstraite]

Une structure de donnée (abstraite) est une collection d'éléments munie d'opérations.

Exemple 24

Par exemple, on peut définir la *liste* comme une collection d'éléments munies des opérations suivantes :

- accéder et modifier un élément par son indice
- ajouter un élément à la fin
- retirer un élément à la fin
- connaître la longueur de la liste

Exemple 25

Par exemple, on peut définir le *dictionnaire* comme une collection d'éléments munies des opérations suivantes :

- accéder, modifier ou insérer une valeur par sa clé
- itérer sur toutes les clés
- connaître le nombre d'éléments dans le dictionnaire

Remarque 26

La *complexité* des opérations dépend de l'*implémentation* de la structure de donnée. Par exemple, la librairie standard de python propose des implémentation de listes et dictionnaires dont toutes les opérations sont en $O(1)$. (en vérité, en $O(1)$ en moyenne, mais ce n'est pas important ici.)

[demo](#)

V.2) Pile

Image : une pile d'assiettes. On peut en poser une par dessus, ou enlever celle du dessus, mais c'est tout !

Définition 27 [Pile]

Une *pile* est une structure de donnée munie des opérations suivantes :

- empiler un élément (l'ajouter en haut de la pile)
- dépiler un élément (prendre l'élément en haut de la pile et le renvoyer)
- (optionnel) regarder l'élément en haut de la pile
- (optionnel) connaître la hauteur de la pile

Pour implémenter une pile de manière efficace, on peut simplement utiliser... une liste ! Avec la correspondance suivante :

- empiler : `P.append(...)` en $O(1)$
- dépiler : `P.pop()` en $O(1)$
- regarder l'élément en haut de la pile : `P[-1]` en $O(1)$
- connaître la hauteur de la pile : `len(P)` en $O(1)$

Remarque 28

En anglais, pile se dit "stack" ou bien "LIFO" pour "Last In First Out", ce qu'on pourrait traduire par "dernier arrivé premier servi".

V.3) File

Image : Une file d'attente. On peut ajouter un élément au début de la file, et enlever un élément à la fin de la file (mais on ne double pas !).

Définition 29 [File]

Une *file* est une structure de donnée munie des opérations suivantes :

- enfiler un élément (l'ajouter au début de la file)
- défiler un élément (prendre l'élément à la fin de la file et le renvoyer)
- (optionnel) regarder l'élément à la fin de la file
- (optionnel) connaître la taille de la file

On peut à nouveau utiliser les listes pour implémenter une file, mais ça ne sera pas très efficace ...

- enfiler : `F.append(...)` en $O(1)$
- défiler : `F.pop(0)` en $O(n)$ (!)
- regarder l'élément à la fin de la file: `F[0]` en $O(1)$
- connaître la hauteur de la file : `len(F)` en $O(1)$

Il est possible d'implémenter une file avec toutes ses opérations en $O(1)$. Cela est par exemple fait dans la structure deque de Python : [demo](#) (Si on attend que vous utilisiez deque, tout vous sera rappelé dans le sujet).

Remarque 30

En anglais, on parle de queue ou de FIFO, “First In First Out”, soit “Premier arrivé premier servi”.

D’ailleurs, deque signifie double-ended queue.

V.4) File de priorité

Image : L’attente à l’hôpital : le plus urgent est traité en premier.

Dans cette collection, on va associer à chaque élément une *priorité* (en général un nombre).

Définition 31 [Pile]

Une *file de priorité* est une structure de donnée munie des opérations suivantes :

- ajouter un élément
- supprimer et renvoyer l’élément de priorité maximale
- (optionnel) regarder l’élément de priorité maximale
- (optionnel) connaître la taille de la file de priorité

Il y a deux implémentations “naives” de files de priorité au programme :

- la liste non triée.
 - ajouter : append $O(1)$
 - supprimer : chercher l’élément de priorité minimale dans la liste, le supprimer. $O(n)$
 - regarder l’élément de priorité maximale : le chercher. $O(n)$
- la liste triée : on maintiens toujours les éléments triés par ordre de priorité croissant.
 - ajouter : insérer l’élément à sa place dans la liste. $O(n)$
 - supprimer : `.pop()`, $O(1)$
 - regarder l’élément de priorité maximale $O(1)$

En pratique, ce ne sont pas de très bonnes implémentations, mais on s’en contentera.

Il peut être utile de savoir qu’il est possible d’implémenter une file de priorité avec toutes les opérations en $O(\log n)$. (Les options verront comment !)

VI) Les parcours

VI.1) Anatomie d’un parcours

On a besoin :

- d’une structure de donnée pour stocker des noeuds

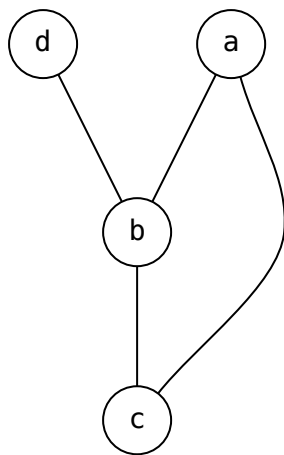
PARCOURS D’UN GRAPHE

Entrée : G un graphe, s un élément de départ

```
1  Ajouter s dans la structure
2  Tant que la structure n’est pas vide
3      Prendre x un élément de la structure
4      (...)
5  Ajouter ses voisins non visités à la structure.
```

VI.2) Parcours en profondeur : la pile

Voici une implem en python, qui imprime simplement les sommets dans l’ordre. On suppose que le graphe est implémenté par liste d’adjacence :



```

G = { "a": ["b", "d"],
      "b": ["d", "a", "c"],
      "d": ["b"],
      "c": ["b", "a"]}

```

```

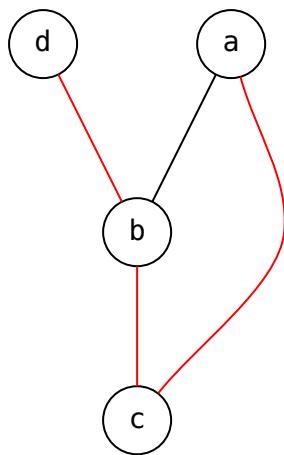
def parcours_profondeur(G, d):
    pile = [d]
    while len(pile) > 0:
        ...
        ...
        x = pile.pop()
        print(x)
        for voisin in G[x]:
            pile.append(voisin)

```

```

def parcours_profondeur(G, d):
    pile = [d]
    deja_vu = {d: True}
    while len(pile) > 0:
        x = pile.pop()
        print(x)
        # On ajoute les voisins à la pile
        for voisin in G[x]:
            if voisin not in deja_vu:
                pile.append(voisin)
                #Evite d'ajouter les mêmes sommets à la pile plusieurs fois
                deja_vu[voisin] = True

```



Version récursive

```

def parcours_profondeur(G, d, deja_vu):
    for voisin in G[x]:
        if voisin not in deja_vu:
            deja_vu[voisin] = True
            parcours_profondeur(G, voisin, deja_vu)
  
```

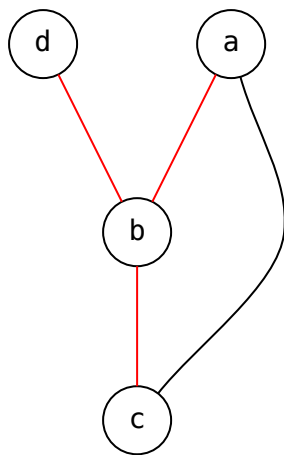
VI.3) Parcours en largeur

```

from collections import deque
  
```

```

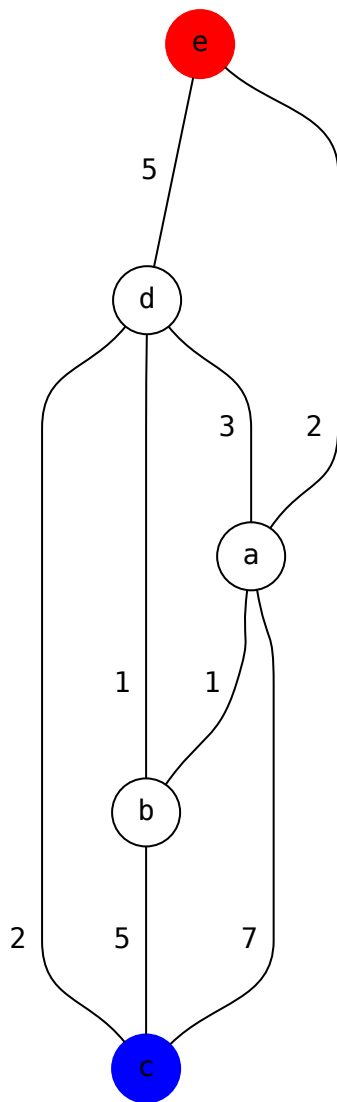
def parcours_largeur(G, d):
    file = deque()
    file.appendleft(d)
    deja_vu = {d: True}
    while len(pile) > 0:
        x = pile.pop()
        print(x)
        # On ajoute les voisins à la pile
        for voisin in G[x]:
            if voisin not in deja_vu:
                pile.appendleft(voisin)
                #Evite d'ajouter les mêmes sommets à la pile plusieurs fois
                deja_vu[voisin] = True
  
```



Pour un crawler web, lequel utiliseriez-vous ? Et pour trouver la sortie d'un labyrinthe ?

VII) Plus court chemin

Sur un graphe non pondéré, pouvez-vous faire un parcours qui permet ça ?

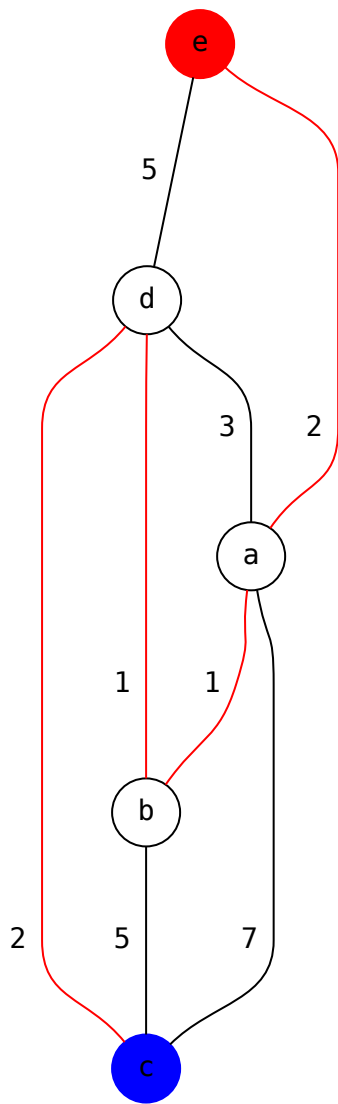


Plan, autoroutes, routes de campagnes, etc

BFS : on augmente la profondeur graduellement : d'abord 1 de distance, puis 2, etc.

Idée : plutôt que d'augmenter la profondeur, on augmente la pondération.

Déroulement sur exemple



VII.1) Algorithme de Dijkstra

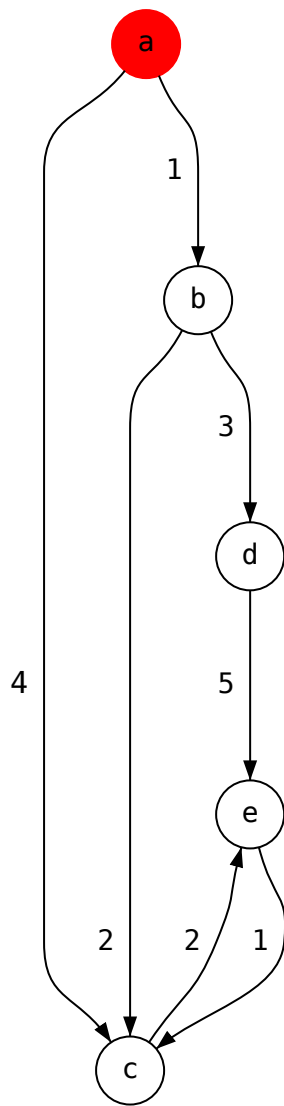
ALGORITHME DE DIJKSTRA

Entrée : G un graphe connexe, d un sommet de départ

Sortie : La distance de d à chaque sommet du graphe.

```
1  Soit F une file de priorité, avec tous les noeuds associé à un poids  $+\infty$ 
2  Soit V un dictionnaire
3  Ajouter (d, 0) à F
4  Tant que F n'est pas vide
5      Prendre (x, l) le couple minimal de F
6      Ajouter (x, l) à V
7      Pour chaque arrête (x, y,  $\rho$ )
8          Si y n'est pas dans V
9              Si y n'est pas dans F
10                 └ ajouter (y, l +  $\rho$ ) F.
11             Sinon
12                 └ mettre à jour le poids de y dans F, au minimum entre le poids actuel et l +  $\rho$ 
13 └ Renvoyer V
```

Dérouler sur l'exemple.



Quelle est la complexité ? ($O((|E| + |S|) \log |S|)$)

En python :

```
import math

def djikstra(G, d):
    """Implemente l'algorithme de Dijkstra sur un graphe par liste d'adjacence.
    Renvoie un dictionnaire qui à chaque sommet associe sa distance à d.
    On utilise une implémentation naive de file de priorité.
    """
    noeuds = {s: math.inf for s in G.keys()}
    noeuds[d] = 0
    V = {}
    while len(noeuds) > 0:
        # On trouve le plus petit élément de noeuds
        sommet_min = None
        distance_min = math.inf
        for (sommet, distance) in noeuds.items():
            if distance < distance_min:
                distance_min = distance
                sommet_min = sommet
        # Si le sommet minimum est None, c'est qu'on a parcouru toute la composante
        # On peut donc renvoyer le dictionnaire;
        if sommet_min is None:
            return V
        # On le supprime de la liste des noeuds et on l'ajoute au dictionnaire
        del noeuds[sommet_min]
        V[sommet_min] = distance_min
        # On met à jour la distance des voisins
        for (voisin, poids) in G[sommet_min]:
            if voisin not in V:
                noeuds[voisin] = min(noeuds[voisin], distance_min + poids)
    return V
```

Quelle est la complexité de cette implémentation ? -> $O(|V|^2)$

<https://visualgo.net/>

VII.2) A*

video

L'idée de l'algorithme A* est d'utiliser une **heuristique**, c'est à dire une indication sur le résultat spécifique au problème traité, souvent facile à calculer mais imprécise.

En l'occurrence, cette heuristique est une indication sur la distance minimum qu'il reste pour parvenir au résultat. Pour que notre algorithme reste correcte, l'heuristique doit toujours être plus petite que la vraie distance !

Exemple : cas d'une carte. On peut prendre la distance "à vol d'oiseau" ! Quelque soit le chemin emprunté, on ne pourra pas aller plus vite.

Supposons qu'on ai une fonction h qui code l'heuristique. Dans ce cas, on ne va plus prendre le sommet de distance à l'origine d minimale, mais celui qui minimise (d + h) ! C'est à dire celui qui a le potentiel d'être sur le chemin le plus court.

La complexité de A^* n'est pas meilleure que djikstra dans le pire cas. Mais dans la majorité des cas réels, cela va beaucoup plus vite !